

FCN - Indice documentale della libreria `fcn_lib_calc`

Documento di riferimento per le dichiarazioni pubbliche presenti nell'header `fcn_lib_calc.h`, con descrizioni tecniche sintetiche e note implementative derivate dalla libreria `fcn_lib_calc.cpp`

Alias, costanti ed enumerazioni

```
using Vec = std::vector<double>
```

Alias per vettori reali in doppia precisione, usato come tipo base per segnali, nodi, colonne e autovalori

```
using VecN = std::vector<int>
```

Alias per vettori interi, utile come contenitore di indici o contatori discreti

```
using Mat = std::vector<Vec>
```

Alias per matrici reali dense rappresentate come vettori di righe

```
using KM = matplot::keyword_manual_type
```

Alias di comodo per keyword manuali di Matplot++

```
using KA = matplot::keyword_automatic_type
```

Alias di comodo per keyword automatiche di Matplot++

```
const double PI = std::acos(-1.0);
```

Costante geometrica π calcolata tramite `acos(-1.0)`

```
enum class SortOrder { Asc, Desc };
```

Enumerazione per il riordino crescente o decrescente di autovalori, valori singolari e colonne associate

funzioni di gestione interfaccia utente

```
cin_clear
```

```
void cin_clear();
```

Pulisce lo stato di `std::cin` in caso di errore e scarta l'eventuale input residuo disponibile nel buffer

```
color_bool
```

```
string color_bool(  
    const bool val  
)
```

Restituisce una stringa ANSI di colore associata a un valore booleano, pensata per evidenziare esiti logici in output testuale

```
color_dbl
```

```
string color_dbl(  
    const double val  
)
```

Restituisce una stringa ANSI di colore in funzione del segno e dell'ordine di grandezza del numero, usando soglie numeriche interne per distinguere valori positivi, negativi e quasi nulli

color_rst

```
void color_rst();
```

Ripristina il colore standard del terminale emettendo il codice ANSI di reset

funzioni di conversione formato ed equivalenza unita' di misura

itostr

```
std::string itostr(  
    const int nn  
);
```

Converte un intero in stringa tramite `std::to_string`

deg2rad

```
double deg2rad(  
    const int alpha  
);
```

Converte un angolo espresso in gradi nel corrispondente valore in radianti usando la costante PI

format_numstr

```
string format_numstr(  
    double v  
);
```

Formatta un numero reale in notazione scientifica con due cifre decimali significative nel mantissa output

funzioni di base per intervalli

h_ticks

```
double h_ticks(  
    const double a_start,  
    const double a_stop,  
    const int a_points  
);
```

Calcola il passo uniforme di una griglia su intervallo chiuso come $(a_stop - a_start)/(a_points - 1)$. Termina l'esecuzione se l'intervallo non è valido

nodi_bubblesort

```
Vec nodi_bubblesort(  
    const Vec x_uns,  
    const int totnum  
);
```

Restituisce una copia ordinata in senso crescente del vettore di ingresso, mediante ordinamento elementare a confronti successivi

nodi_equidistanti

```
Vec nodi_equidistanti(  
    const double amin,  
    const double amax,  
    const int NPoints  
);
```

Genera `NPoints` nodi equidistanti nell'intervallo `[amin, amax]` usando il passo calcolato da `h_ticks`

nodis_random

```
Vec nodis_random(  
    const double amin,  
    const double amax,  
    const int NPoints  
);
```

Genera un vettore di nodi pseudo-casuali scalando campioni prodotti da `std::mt19937` sull'intervallo richiesto

funzioni matematiche ad uso callback

f_x

```
double f_x(  
    double t  
);
```

Callback identità, restituisce il valore `t`

f_x1

```
double f_x1(  
    double t  
);
```

Callback identità traslata, restituisce `t - 1`. Usata come coefficiente `at(t)` nei sistemi di Cauchy con termine lineare non omogeneo

at_zero

```
double at_zero(  
    double t  
);
```

Callback costante nulla per il coefficiente di `z(t)` nel sistema di Cauchy: restituisce `0.0`

at_one

```
double at_one(  
    double t  
);
```

Callback costante unitaria per il coefficiente di `z(t)` nel sistema di Cauchy: restituisce `1.0`. Usata come default in `matrix_build_cauchy`

ft_zero

```
double ft_zero(  
    double t  
);
```

Callback sorgente nulla: restituisce `0.0`. Usata come termine forzante di default in `matrix_build_cauchy`

f_sin

```
double f_sin(  
    double t  
);
```

Callback seno, restituisce `sin(t)`

f_cos

```
double f_cos(  
    double t  
);
```

Callback coseno, restituisce $\cos(t)$

f_atan

```
double f_atan(  
    double t  
);
```

Callback arcotangente, restituisce $\text{atan}(t)$

f_atan_d

```
double f_atan_d(  
    double t  
);
```

Callback derivata dell'arcotangente, restituisce $1/(1+t^2)$

f_sin2plus1

```
double f_sin2plus1(  
    double t  
);
```

Callback che restituisce $1/(1+\sin^2(t))$

fcallb

```
double fcallb(  
    double t,  
    double (*f)(double)  
);
```

Wrapper generico per valutare una callback reale **f** nel punto **t**

funzioni macro algebra lineare

linear_subst_BW

```
Vec linear_subst_BW(  
    const Mat &U,  
    const Vec &y  
);
```

Risolve un sistema triangolare superiore $Ux = y$ per sostituzione all'indietro

Note implementative - Implementazione didattica $O(n^2)$ senza pivoting né controlli strutturali sulla triangularità
- La routine assume elementi diagonali non nulli; è quindi adatta a fattorizzazioni già validate a monte

linear_subst_FW

```
Vec linear_subst_FW(  
    const Mat &L,  
    const Vec &b  
);
```

Risolve un sistema triangolare inferiore $Lx = b$ per sostituzione in avanti

Note implementative - Algoritmo classico $O(n^2)$ su matrice inferiore con diagonale non nulla - Non sono previsti pivoting o controlli di degenerazione oltre alla divisione diretta sui pivot

linear_jacobi_autoval_simmetrica

```
void linear_jacobi_autoval_simmetrica(  
    const Mat& A,  
    Vec& lambda,  
    Mat& V  
);
```

Calcola autovalori e autovettori di una matrice simmetrica reale mediante iterazioni di Jacobi con annullamento progressivo dei termini fuori diagonale

Note implementative - La matrice viene copiata localmente e diagonalizzata tramite rotazioni piane, aggiornando simultaneamente la matrice degli autovettori V - Il criterio d'arresto usa il massimo elemento fuori diagonale confrontato con una tolleranza interna $1e-12$, con limite massimo di $100 * n * n$ iterazioni

linear_LU_calcola_autovalori

```
Vec linear_LU_calcola_autovalori(  
    const Mat& A  
);
```

Restituisce una stima degli autovalori di A come elementi diagonali del fattore U ottenuto da una fattorizzazione LU

Note implementative - È una procedura euristica e non generale; il sorgente stesso la considera adatta soprattutto a matrici simmetriche e positive semidefinite in contesti sperimentali - L'accuratezza dipende fortemente dall'assenza di pivoting e dalla buona condizione della matrice

linear_LU_dec

```
void linear_LU_dec(  
    const Mat &A,  
    Mat &L,  
    Mat &U  
);
```

Calcola una fattorizzazione LU senza pivoting della matrice quadrata A, producendo L unitaria inferiore e U triangolare superiore

Note implementative - La procedura modifica U in place e costruisce L tramite moltiplicatori elementari di eliminazione gaussiana - Se incontra un pivot nullo, solleva eccezione perché il pivoting non è implementato

linear_LU_inversa

```
Mat linear_LU_inversa(  
    const Mat& A  
);
```

Calcola l'inversa di A risolvendo, colonna per colonna, i sistemi $Ax = e_j$ tramite la fattorizzazione LU

Note implementative - Ogni colonna dell'inversa viene ottenuta con una doppia sostituzione triangolare a partire dal versore canonico corrispondente - Il metodo è chiaro e adatto alla documentazione didattica, ma eredita i limiti della LU senza pivoting

linear_LU_risolve_colonna

```
Vec linear_LU_risolve_colonna(  
    const Mat& L,
```

```

    const Mat& U,
    Vec x
);

```

Risolve un sistema già fattorizzato LU, applicando prima la sostituzione in avanti e poi quella all'indietro

Note implementative - La firma accetta il termine noto per valore, ma la routine lo usa concettualmente come vettore generico del secondo membro - Nel sorgente sono presenti stampe di debug dei vettori intermedi y e x

linear_LU_risolve_sistema

```

Vec linear_LU_risolve_sistema(
    const Mat& T,
    const Vec& x
);

```

Risolutore ad alto livello che fattorizza T in LU e risolve il sistema lineare con termine noto x

linear_max_autoval_pwr_any_res

```

double linear_max_autoval_pwr_any_res(
    const Mat& M,
    int max_iter,
    double tol
);

```

Stima il massimo autovalore di $M^T M$ senza costruirlo esplicitamente, usando un metodo delle potenze con controllo di convergenza basato sulla variazione della norma del vettore iterato

Note implementative - Il vettore iniziale è casuale gaussiano e viene normalizzato in norma 2 - Il metodo lavora con prodotti successivi $A*q$ e $A^T*(A*q)$, evitando la formazione esplicita di $M^T M$. tol controlla il criterio d'arresto, max_iter il numero massimo di iterazioni

linear_max_autoval_pwr_any

```

double linear_max_autoval_pwr_any(
    const Mat& M,
    int max_iter,
    double tol
);

```

Stima il massimo autovalore di $M^T M$ tramite metodo delle potenze, con quoziente di Rayleigh valutato sull'iterato normalizzato

Note implementative - Anche questa variante evita la costruzione esplicita di $M^T M$, ma usa una logica di arresto sulla differenza tra stime successive dell'autovalore - È utile quando interessa una stima della norma spettrale di una matrice rettangolare tramite $\|M\|_2 = \sqrt{\lambda_{\max}(M^T M)}$

linear_max_autoval_pwr_AtA

```

double linear_max_autoval_pwr_AtA(
    const Mat& M,
    int max_iter,
    double tol
);

```

Applica il metodo delle potenze a una matrice quadrata già interpretata come $A^T A$, restituendo una stima del suo autovalore dominante

Note implementative - Il vettore iniziale è uniforme e normalizzato - La procedura è più semplice concettualmente, ma richiede che la matrice di input sia già stata costruita esplicitamente

funzioni per la gestione di vettori e matrici

matrix_build_cauchy

```
void matrix_build_cauchy(  
    int n,  
    double t0,  
    double T,  
    double z_bc,  
    Vec &t,  
    Vec &avals,  
    Vec &fvals,  
    Mat &L,  
    Vec &b,  
    double (*at)(double) = at_one,  
    double (*ft)(double) = ft_zero,  
    bool backw = false  
);
```

Costruisce il sistema lineare $Lx = b$ associato a un problema di Cauchy lineare scalare del primo ordine con coefficiente $at(t)$ e termine forzante $ft(t)$, cioè nella forma $z'(t) = a(t) * z(t) + f(t)$ discretizzato su una griglia di n nodi nell'intervallo $[t0, T]$

La routine costruisce: - il vettore dei nodi t ; - il vettore $avals$ dei campioni del coefficiente $a(t)$; - il vettore $fvals$ dei campioni del termine noto $f(t)$; - la matrice bidiagonale L ; - il termine noto b .

Signature

```
void matrix_build_cauchy(  
    int n,  
    double t0,  
    double T,  
    double z_bc,  
    Vec &t,  
    Vec &avals,  
    Vec &fvals,  
    Mat &L,  
    Vec &b,  
    double (*at)(double),  
    double (*ft)(double),  
    bool backw  
);
```

Parametri

- n : numero di nodi della discretizzazione.
- $t0$: estremo iniziale dell'intervallo.
- T : estremo finale dell'intervallo.
- z_bc : valore assegnato al bordo, iniziale o finale a seconda del verso di integrazione.
- t : vettore dei nodi equispaziati.
- $avals$: campioni del coefficiente $a(t)$ sui nodi.
- $fvals$: campioni del termine noto $f(t)$ sui nodi.
- L : matrice del sistema lineare discreto.
- b : termine noto del sistema lineare discreto.
- at : puntatore a funzione per il coefficiente $a(t)$; se `nullptr`, viene sostituito con una funzione costante unitaria.
- ft : puntatore a funzione per il termine noto $f(t)$; se `nullptr`, viene sostituito con una funzione costante nulla.
- $backw$: selettore del verso di costruzione; `false` per schema forward, `true` per schema backward.

Convenzioni sulle callback

La routine assume callback con signature compatibile `double(double)`

Sono previste anche funzioni costanti di supporto come:

```
double at_zero(double /*t*/) { return 0.0; }
double at_one(double /*t*/) { return 1.0; }
double ft_zero(double /*t*/) { return 0.0; }
```

In particolare: - `at_zero` realizza il caso di integrazione pura $a(t) = 0$; - `at_one` realizza il caso $a(t)=1$; - `ft_zero` realizza il caso omogeneo $f(t)=0$.

Schema forward

Nel caso `backw == false`, la routine costruisce un sistema bidiagonale inferiore associato alla discretizzazione forward del problema con dato iniziale.

La riga interna del sistema è

$$z_i - (1 + h a(t_{i-1})) z_{i-1} = h f(t_{i-1}), \quad i = 1, \dots, n-1$$

mentre la prima riga impone il dato iniziale

$$z_0 = z_{bc}.$$

In questo caso la matrice è triangolare inferiore e può essere risolta con `linear_subst_FW(...)`.

Schema backward

Nel caso `backw == true`, la routine costruisce un sistema bidiagonale superiore associato alla discretizzazione backward del problema con dato finale.

La riga interna del sistema è

$$(-1 - h a(t_i)) z_i + z_{i+1} = h f(t_i), \quad i = 0, \dots, n-2$$

mentre l'ultima riga impone il dato finale

$$z_{n-1} = z_{bc}$$

In questo caso la matrice è triangolare superiore e può essere risolta con `linear_subst_BW(...)`.

Casi particolari

La stessa routine copre automaticamente diversi casi notevoli: - **integrazione pura**: $z'(t) = f(t)$, ottenuta con $a(t) = 0$; - **equazione omogenea**: $z'(t) = a(t)z(t)$, ottenuta con $f(t) = 0$; - **caso separabile testato**: $z'(t) = z(t) + f(t)$, ottenuto con $a(t) = 1$.

In questo modo la distinzione tra i diversi tipi di ODE non è affidata a funzioni pubbliche differenti, ma emerge direttamente dalla formula discreta e dai campioni delle callback `at` e `ft`.

Note implementative

La routine inizializza internamente i vettori `t`, `avals`, `fvals`, la matrice `L` e il vettore `b`, così da evitare dipendenze da contenuti residui già presenti nei contenitori passati per riferimento

Il passo di discretizzazione è

$$h = \frac{T - t_0}{n - 1}.$$

I nodi sono quindi costruiti come

$$t_i = t_0 + i h, \quad i = 0, \dots, n - 1.$$

Questa scelta mantiene una corrispondenza diretta tra nodo, campionamento dei coefficienti e riga discreta del sistema lineare.

matrix_build_cauchy_int

```
void matrix_build_cauchy_int(
    int n,
    double t0,
    double T,
    double x0,
    Vec &t,
    Vec &fvals,
    Mat &L,
    Vec &b,
    double (*ft)(double),
    bool backw
);
```

Versione semplificata di **matrix_build_cauchy** con coefficiente **at(t)** = 0 (equazione di integrazione pura **z'** = **f(t)**). Campiona **ft** internamente e costruisce il sistema bidiagonale corrispondente. Deprecata dopo il collaudo di **matrix_build_cauchy**

Note implementative - Caso limite con solo termine forzante; equivalente a un'integrazione numerica di **f(t)** con schema alle differenze finite

matrix_build_cauchy_sep

```
void matrix_build_cauchy_sep(
    int n,
    double t0,
    double T,
    double x0,
    Vec &t,
    Vec &fvals,
    Mat &L,
    Vec &b,
    double (*ft)(double),
    bool backw
);
```

Variante per ODE a variabili separabili (**ft** = 0): costruisce il sistema omogeneo **z' - a(t)z = 0**. Utile come banco di test per confronto con la soluzione esatta analitica **z(t) = z_bc * exp(a(s)ds)**. Deprecata dopo il collaudo di **matrix_build_cauchy**

matrix_build_derivata1

```
Mat matrix_build_derivata1(
    int n,
    double a,
    double b
);
```

Costruisce una matrice delle differenze finite in avanti di dimensione $(n-1) \times n$ per approssimare la derivata prima su una griglia uniforme

Note implementative - Gli elementi non nulli sono $-1/h$ sulla diagonale e $1/h$ sulla sopradiagonale immediata - Il passo h è ottenuto da `h_ticks(a, b, n - 1)`

`matrix_build_gausskernel`

```
Mat matrix_build_gausskernel(  
    const int N,  
    const double sigma,  
    const double h,  
    bool norm_flag,  
    Vec& Indicatori  
);
```

Costruisce una matrice kernel gaussiana campionata su una griglia uniforme, con possibilità di normalizzazione per righe e restituzione di indicatori numerici

Note implementative - L'elemento K_{ij} è ottenuto da una gaussiana centrata sulla differenza $x_i - x_j$ - Se `norm_flag` è attivo, le righe vengono normalizzate. Il vettore `Indicatori` viene riempito con quantità diagnostiche legate alla norma della matrice, della sua inversa e al numero di condizionamento approssimato

`matrix_build_gram`

```
Mat matrix_build_gram(  
    const Vec& x,  
    const int K  
);
```

Costruisce una matrice di Gram $K \times K$ campionando una famiglia di funzioni sui nodi x e usando prodotti scalari tra campioni

Note implementative - Nel sorgente i campioni sono ottenuti con `vector_campiona_f_k(..., f_cos)` e la matrice viene riempita in modo simmetrico - La routine è utile come banco di prova per ortogonalità e dipendenza lineare di basi discrete

`matrix_build_Id`

```
Mat matrix_build_Id(  
    int n  
);
```

Costruisce la matrice identità $n \times n$

`matrix_build_triang`

```
Mat matrix_build_triang(  
    int n  
);
```

Costruisce una matrice triangolare inferiore piena di uni

`matrix_build_triang_inv`

```
Mat matrix_build_triang_inv(  
    int n  
);
```

Costruisce una matrice triangolare inferiore che rappresenta l'inversa della matrice triangolare di uni del caso precedente, con 1 in diagonale e -1 sulla sottodiagonale immediata

matrix_build_zero

```
Mat matrix_build_zero(  
    int righe,  
    int colonne  
);
```

Costruisce una matrice nulla delle dimensioni richieste

matrix_calcola_deriv_byiter

```
Vec matrix_calcola_deriv_byiter(  
    const Vec &u,  
    const double a,  
    const double b  
);
```

Approssima la derivata del vettore u campionato sull'intervallo $[a, b]$ tramite differenze finite iterative, senza costruire esplicitamente la matrice delle differenze

matrix_calcola_deriv_bymatr

```
Vec matrix_calcola_deriv_bymatr(  
    const Mat &D,  
    const Vec &u  
);
```

Calcola il prodotto matrice-vettore $D * u$ dove D è la matrice delle differenze finite pre-costruita (tipicamente da `matrix_build_derivata1`). Approccio matriciale equivalente a `matrix_calcola_deriv_byiter`

Note implementative - Le due varianti (`byiter` e `bymatr`) producono risultati identici sulla stessa griglia e sono mantenute entrambe come confronto didattico tra approccio iterativo e approccio matriciale

matrix_calcola_errore_Fr

```
double matrix_calcola_errore_Fr(  
    const Mat& A,  
    const Mat& B  
);
```

Calcola la norma di Frobenius della differenza $A - B$

matrix_calcola_media

```
void matrix_calcola_media(  
    const Mat& A,  
    Vec& avg_col,  
    Vec& avg_row  
);
```

Calcola le medie per colonna e per riga della matrice A

matrix_calcola_norma

```
double matrix_calcola_norma(  
    int norma,  
    const Mat& A  
);
```

Calcola varie norme matriciali e alcune stime della norma 2, in funzione del codice intero `norma`

Note implementative - `norma = 1` restituisce la norma 1, `norma = 0` la norma infinito, `norma = -1` la norma di Frobenius - `norma = 2, 12 e 22` attivano diverse strategie per stimare la norma 2, basate rispettivamente su LU

sperimentale o su metodi delle potenze con o senza costruzione esplicita di $A^T A$ - La presenza di più codifiche rende utile documentare esplicitamente il significato operativo del parametro nelle note a margine del documento finale

matrix_centra_su_media

```
Mat matrix_centra_su_media(  
    const Mat& A,  
    const Vec& avg_vec,  
    bool by_col = true  
);
```

Centra la matrice sottraendo a ciascun elemento la media di colonna o di riga, a seconda del flag **by_col**

matrix_differenza_dump

```
Mat matrix_differenza_dump(  
    const Mat& A,  
    const Mat& B  
);
```

Restituisce la differenza $A - B$ se le dimensioni sono compatibili; in caso contrario termina il programma con messaggio diagnostico

matrix_dump

```
void matrix_dump(  
    const Mat &A,  
    const std::string &nome  
);
```

Stampa in console una matrice formattata, con colorazione degli elementi in funzione del loro segno e ordine di grandezza

matrix_estende_ridotta

```
Mat matrix_estende_ridotta(  
    const Mat& A,  
    int n,  
    bool bycol  
);
```

Riduce o mantiene una matrice alle dimensioni richieste, tagliando righe o colonne a seconda del flag **bycol**

matrix_normalize_byrow

```
Mat matrix_normalize_byrow(  
    Mat& K  
);
```

Restituisce una copia della matrice in cui ogni riga è normalizzata rispetto alla propria somma degli elementi

matrix_ordina_diagonale

```
void matrix_ordina_diagonale(  
    Vec& lambda,  
    Mat& V,  
    double zero_tol = 0.0,  
    SortOrder order = SortOrder::Desc  
);
```

Ordina il vettore `lambda` e trascina coerentemente le colonne della matrice `V`, con possibile azzeramento preliminare dei valori inferiori alla soglia `zero_tol`

Note implementative - La routine è centrale nella pipeline SVD/autovalori, perché consente di riordinare autovalori e autovettori mantenendo l'allineamento colonnare - Il parametro `order` distingue ordinamento crescente e decrescente

`matrix_ortogonalizza_GSmod`

```
void matrix_ortogonalizza_GSmod(
    Mat& Q,
    int j0 = 0
);
```

Ortonormalizza le colonne di `Q` con Gram-Schmidt modificato, a partire dalla colonna `j0`

Note implementative - Serve sia come routine autonoma sia come supporto al completamento di basi ridotte in contesti SVD - Le colonne quasi nulle vengono lasciate a zero anziché generare errore

`matrix_prodotto_coeff`

```
Mat matrix_prodotto_coeff(
    const Mat& A,
    const double coeff
);
```

Restituisce il prodotto scalare `coeff * A`

`matrix_prodotto_AtA`

```
Mat matrix_prodotto_AtA(
    const Mat& A,
    bool A_right = true
);
```

Costruisce $A^T A$ se `A_right` è `true`, oppure $A A^T$ se `A_right` è `false`

`matrix_prodotto_matrix`

```
Mat matrix_prodotto_matrix(
    const Mat& A,
    const Mat& B
);
```

Calcola il prodotto matrice-matrice $A * B$, con controllo di compatibilità dimensionale e lancio di eccezione in caso di mismatch

`matrix_prodotto_vector`

```
Vec matrix_prodotto_vector(
    const Mat& A,
    const Vec& v
);
```

Calcola il prodotto matrice-vettore $A * v$, con controllo di compatibilità dimensionale

`matrix_test_ortogonale`

```
void matrix_test_ortogonale(
    const Mat& A,
    string s
);
```

Esegue un test di ortogonalità stampando il prodotto $A^T A$ associato alla matrice A

matrix_trasposta

```
Mat matrix_trasposta(  
    const Mat& A  
);
```

Restituisce la trasposta della matrice A

vector_add_noise

```
Vec vector_add_noise(  
    Vec& v,  
    double e  
);
```

Aggiunge rumore casuale al vettore v , con ampiezza proporzionale alla sua norma e al parametro e espresso in millesimi

vector_build_versore_canonico

```
Vec vector_build_versore_canonico(  
    int j,  
    int N  
);
```

Costruisce il versore canonico e_j di dimensione N

vector_calcola_norma

```
double vector_calcola_norma(  
    int norma,  
    const Vec& V  
);
```

Calcola la norma 2 o la norma infinito del vettore, a seconda del codice `norma`

vector_campiona_f

```
Vec vector_campiona_f(  
    const Vec &x,  
    double (*ft)(double)  
);
```

Campiona una funzione reale `ft` sui nodi contenuti in x

vector_campiona_f_k

```
Vec vector_campiona_f_k(  
    int k,  
    const Vec &x,  
    double (*ft)(double)  
);
```

Campiona una funzione base parametrica sui nodi x , usando k come fattore di frequenza o parametro discreto di famiglia

Note implementative - Nel sorgente la callback viene richiamata come `fcallb(k * x[i], ft)`, quindi k agisce come acceleratore della variabile di input

vector_dump

```
void vector_dump(  
    Vec x,  
    int colspan,  
    int totnum,  
    const std::string s  
);
```

Stampa in console il contenuto del vettore con impaginazione a colonne, utile per debug e ispezione manuale

vector_householder_bycol

```
Vec vector_householder_bycol(  
    const Vec& v  
);
```

Costruisce il vettore di Householder associato a una colonna, in modo che la riflessione annulli tutti gli elementi sotto il primo

Note implementative - Se il vettore ha norma nulla, restituisce il primo versore canonico come trasformazione neutra - La convenzione di segno è scelta per aumentare la stabilità numerica nella costruzione di $u = v + \text{sign}(v_0) ||v|| e_1$

vector_householder_byrow

```
Vec vector_householder_byrow(  
    const Vec& v  
);
```

Versione per righe del costruttore di Householder; nel codice delega direttamente alla versione per colonne

vector_prodotto_scalare

```
double vector_prodotto_scalare(  
    const Vec &u,  
    const Vec &v  
);
```

Calcola il prodotto scalare euclideo tra due vettori della stessa dimensione

vector_reverse

```
Vec vector_reverse(  
    const Vec& v  
);
```

Restituisce una copia del vettore con ordine degli elementi invertito

vector_segnale_finestra

```
Vec vector_segnale_finestra(  
    int N,  
    double a,  
    double b,  
    double t  
);
```

Genera un segnale finestrato di lunghezza N, con valore t nell'intervallo relativo [a, b] e zero altrove

`vector_shift`

```
Vec vector_shift(  
    const Vec &v,  
    const double shift  
);
```

Restituisce una copia del vettore traslata di una quantità costante `shift`

`vector_to_matrix`

```
Mat vector_to_matrix(  
    const Vec& v,  
    bool transp  
);
```

Converte un vettore in matrice colonna oppure in matrice riga, a seconda del flag `transp`

`vector_to_matrix_diag`

```
Mat vector_to_matrix_diag(  
    const Vec& s,  
    int m,  
    int n  
);
```

Costruisce una matrice diagonale $m \times n$ usando gli elementi del vettore `s` sulla diagonale principale fino alla minima dimensione disponibile

funzioni specifiche di trasformazione matrici

`trmatrix_bidiagonalizza`

```
void trmatrix_bidiagonalizza(  
    const Mat& A,  
    Mat& U0,  
    Mat& B,  
    Mat& V0,  
    bool sup_diag = false,  
    bool dump_flag = false  
);
```

Wrapper generale per la bidiagonalizzazione di una matrice rettangolare, con restituzione di fattori ortogonali `U0`, `V0` e matrice bidiagonale `B` tali che $A = U0 B V0^T$

Note implementative - La routine distingue tra caso “wide” e caso “tall”; nel secondo usa la trasposta e riconduce il problema alle funzioni per matrici wide - Il parametro `sup_diag` seleziona bidiagonale superiore o inferiore, mentre `dump_flag` abilita stampe diagnostiche intermedie

`trmatrix_bidiag_wide_to_lower`

```
void trmatrix_bidiag_wide_to_lower(  
    const Mat& X,  
    Mat& U0,  
    Mat& B,  
    Mat& V0,  
    bool dump_flag  
);
```

Riduce una matrice wide a forma bidiagonale inferiore tramite una sequenza di riflessioni di Householder applicate prima a destra e poi a sinistra

Note implementative - Gli aggiornamenti di U0 e V0 sono espliciti e mantengono memoria del prodotto cumulativo delle trasformazioni ortogonali - La funzione è pensata anche come strumento didattico, perché `dump_flag` consente di seguire passo passo struttura della matrice e ortogonalità dei fattori

trmatrix_bidiag_wide_to_upper

```
void trmatrix_bidiag_wide_to_upper(
    const Mat& X,
    Mat& U0,
    Mat& B,
    Mat& V0,
    bool dump_flag
);
```

Riduce una matrice wide a forma bidiagonale superiore applicando Householder su sotto-colonne e sotto-righe in successione tipo Golub–Kahan

Note implementative - La procedura usa la formula efficiente $HA = A - 2w(w^T A)$ e aggiorna i blocchi attivi della matrice senza costruire esplicitamente l'intera riflessione - Anche in questo caso `dump_flag` abilita il tracciamento dei passaggi e dei test di ortogonalità di U0 e V0

trmatrix_SVDQR

```
void trmatrix_SVDQR(
    const Mat& B,
    Mat& Ub,
    Mat& Vb,
    Vec& sigma
);
```

Wrapper ad alto livello per la SVD della matrice B, che richiama la versione ridotta e completa quando necessario la base destra

Note implementative - La funzione mantiene l'interfaccia compatibile con la routine SVD precedente, riordinando i parametri di uscita e completando Vb a base integrale quando la decomposizione nasce in forma ridotta

trmatrix_SVDQR_ridotta

```
void trmatrix_SVDQR_ridotta(
    const Mat& B,
    Mat& Ub,
    Vec& sigma,
    Mat& Vb_red,
    double ev_tol = 1e-12
);
```

Calcola una SVD ridotta di B mediante diagonalizzazione di $B^T B$, ordinamento degli autovalori e ricostruzione dei vettori singolari sinistri da $u_i = (1/\sigma_i)Bv_i$

Note implementative - Gli autovalori di $B^T B$ sono calcolati da `linear_jacobi_autoval_simmetrica`, poi ordinati con `matrix_ordina_diagonale` - Il parametro `ev_tol` serve a schiacciare a zero autovalori negativi di origine numerica o quasi nulli, evitando artefatti nel calcolo di `sqrt(lambda)` - La procedura produce `sigma` e `Vb_red` in forma ridotta; eventuali completamenti ortogonali vengono demandati al livello superiore

trmatrix_test_sv_autoval

```
void trmatrix_test_sv_autoval(
    Vec lambda,
    const Vec& sigma
);
```

Confronta autovalori e valori singolari stampando gli scarti tra `lambda` e `sigma^2`, oltre al confronto tra `sigma` e `sqrt(lambda)`

Funzioni per `matplot++`

`matplot_legend_align`

```
void matplot_legend_align(  
    legend_handle lg,  
    int pos_enum,  
    float xscale,  
    float yscale  
);
```

Allinea e riposiziona la legenda Matplot++ secondo alcune configurazioni discrete codificate da `pos_enum`

Note implementative - Le opzioni implementate includono posizionamento centrato superiore esterno, in basso a destra, al centro a sinistra e in alto a sinistra

`matplot_table_init`

```
figure_handle matplot_table_init(  
    const bool ahold,  
    const std::string &nome,  
    const std::string &titolo,  
    const int xlab,  
    const int ylab  
);
```

Inizializza una figura Matplot++ con layout tiled, dimensioni fissate, nome finestra e titolo configurato

gestione menu principale

`struct MenuItem`

Struttura dati che rappresenta un elemento di menu con chiave numerica, etichetta, azione associata e flag di abilitazione

`struct MenuConfig`

Struttura dati che rappresenta una configurazione di menu completa, con titolo e lista di `MenuItem`

`load_menu_config`

```
MenuConfig load_menu_config(  
    const std::string& filename  
);
```

Carica una configurazione di menu da file JSON e costruisce la struttura `MenuConfig` corrispondente

Note implementative - La routine usa `nlohmann::json`, verifica la presenza dell'array `items` e lancia eccezioni in caso di file non accessibile o struttura non valida - Nel sorgente sono presenti messaggi di debug su working directory e dimensione del file letto

`find_menu_item`

```
const MenuItem* find_menu_item(  
    const MenuConfig& menu,  
    int key  
);
```

Restituisce un puntatore all'elemento di menu con chiave `key`, oppure `nullptr` se non trovato

```
wait_return_to_menu
```

```
void wait_return_to_menu(  
    bool bypass_waitakey  
);
```

Attende il tasto INVIO prima del ritorno al menu, salvo disattivazione esplicita tramite `bypass_waitakey`